SQL Clauses Notes

1. WHERE Clause

Definition

The `WHERE` clause is used to filter rows from a table based on a condition.

It selects only those records that satisfy the specified condition.

Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Important Points

- Used to filter rows.
- Works before grouping.
- Can be used with:
- SELECT
- UPDATE
- DELETE
- Supports operators like:
- `=`
- `>`
- `<`
- `>=`
- `<=`
- `!=`
- `AND`
- `OR`
- `NOT`
- `IN`
- `BETWEEN`
- `LIKE`

Example Table: Students

student_id	name	age	marks	department
1	Arun	20	85	CSE
2	Ravi	21	72	ECE
3	Priya	19	91	CSE
4	Kiran	22	65	EEE
5	Meena	20	88	ECE

Example 1: Simple WHERE

```
SELECT *
FROM Students
WHERE marks > 80;
```

Output

name	marks
Arun	85
Priya	91
Meena	88

Example 2: WHERE with AND

```
SELECT *
FROM Students
WHERE department = 'CSE' AND marks > 80;
```

Output

name	department	marks
Arun	CSE	85
Priya	CSE	91

Example 3: WHERE with LIKE

```sql
SELECT *
FROM Students
WHERE name LIKE 'P%';
```

Output

name
Priya

---

2. GROUP BY Clause

Definition

The `GROUP BY` clause is used to group rows that have the same values into summary rows.

It is mostly used with aggregate functions such as:

- `COUNT()`
- `SUM()`
- `AVG()`
- `MAX()`
- `MIN()`

Syntax

```sql
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Important Points

- Groups rows with similar values.
- Used with aggregate functions.
- Every non-aggregated column in SELECT should be present in GROUP BY.

- Executes after WHERE clause.

Example 1: Count Students in Each Department

```sql
SELECT department, COUNT(*) AS total_students
FROM Students
GROUP BY department;
```

Output

department	total_students
CSE	2
ECE	2
EEE	1

Example 2: Average Marks by Department

```sql
SELECT department, AVG(marks) AS average_marks
FROM Students
GROUP BY department;
```

Output

department	average_marks
CSE	88
ECE	80
EEE	65

Example 3: Maximum Marks in Each Department

```sql
SELECT department, MAX(marks) AS highest_marks
FROM Students
GROUP BY department;
```

Output

department	highest_marks
CSE	91
ECE	88
EEE	65

---

3. HAVING Clause

Definition

The `HAVING` clause is used to filter grouped data.

While `WHERE` filters individual rows, `HAVING` filters groups created using `GROUP BY`.

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Difference Between WHERE and HAVING

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions directly	Can use aggregate functions

---

Example 1: Departments Having More Than 1 Student

```
SELECT department, COUNT(*) AS total_students
FROM Students
```

```
GROUP BY department
HAVING COUNT(*) > 1;
```

Output

department	total_students
CSE	2
ECE	2

Example 2: Departments with Average Marks Greater Than 80

```
SELECT department, AVG(marks) AS avg_marks
FROM Students
GROUP BY department
HAVING AVG(marks) > 80;
```

Output

department	avg_marks
CSE	88

Example 3: Using WHERE and HAVING Together

```
SELECT department, COUNT(*) AS total_students
FROM Students
WHERE marks > 70
GROUP BY department
HAVING COUNT(*) >= 2;
```

Explanation

1. WHERE filters rows with marks greater than 70.
2. GROUP BY groups departments.
3. HAVING filters groups having at least 2 students.

---

4. ORDER BY Clause

Definition

The `ORDER BY` clause is used to sort the result set.

Sorting can be:

- Ascending (`ASC`) → Default
- Descending (`DESC`)

Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column_name ASC;
```

OR

```
SELECT column1, column2
FROM table_name
ORDER BY column_name DESC;
```

Important Points

- `ASC` sorts from smallest to largest.
- `DESC` sorts from largest to smallest.
- Can sort numbers, strings, and dates.
- Multiple columns can be used.

Example 1: Sort by Marks Ascending

```
SELECT *
FROM Students
ORDER BY marks ASC;
```

Output

name	marks
Kiran	65
Ravi	72
Arun	85
Meena	88
Priya	91

Example 2: Sort by Marks Descending

```sql
SELECT *
FROM Students
ORDER BY marks DESC;
```

Output

name	marks
Priya	91
Meena	88
Arun	85
Ravi	72
Kiran	65

Example 3: Multiple Column Sorting

```sql
SELECT *
FROM Students
ORDER BY department ASC, marks DESC;
```

Explanation

- First sorts departments alphabetically.
- Inside each department, marks are sorted in descending order.

SQL Execution Order

Even though SQL queries are written in one order, they execute internally in another order.

Actual Execution Order

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Combined Example

```sql
SELECT department, AVG(marks) AS avg_marks
FROM Students
WHERE marks > 70
GROUP BY department
HAVING AVG(marks) > 80
ORDER BY avg_marks DESC;
```

Step-by-Step Working

Step 1: WHERE

Filters students having marks greater than 70.

name	marks	department
Arun	85	CSE
Ravi	72	ECE
Priya	91	CSE
Meena	88	ECE

Step 2: GROUP BY

Groups remaining students by department.

Step 3: AVG()

Calculates average marks.

department	avg_marks
CSE	88
ECE	80

Step 4: HAVING

Keeps only groups with average marks greater than 80.

department	avg_marks
CSE	88

Step 5: ORDER BY

Sorts the result in descending order.

Final Difference Summary

Clause	Purpose	Works On	Used With
WHERE	Filters rows	Individual rows	SELECT, UPDATE, DELETE
GROUP BY	Groups rows	Multiple rows	Aggregate Functions
HAVING	Filters grouped rows	Groups	GROUP BY
ORDER BY	Sorts results	Final result set	SELECT

Short Interview Points

WHERE

- Filters rows before grouping.
- Cannot directly use aggregate functions.

GROUP BY

- Creates groups of similar rows.
- Commonly used with COUNT, AVG, SUM.

HAVING

- Filters grouped data.
- Used after GROUP BY.

ORDER BY

- Sorts output.
- ASC is default.

Practice Queries

1. Find students scoring above 75

```sql
SELECT *
FROM Students
WHERE marks > 75;
```

---

2. Count students department-wise

```sql
SELECT department, COUNT(*)
FROM Students
GROUP BY department;
```

---

3. Departments having more than 2 students

```sql
SELECT department, COUNT(*)
FROM Students
GROUP BY department
HAVING COUNT(*) > 2;
```

---

4. Sort students by age descending

```sql
SELECT *
FROM Students
ORDER BY age DESC;
```

Conclusion

- `WHERE` filters rows.
- `GROUP BY` groups rows.
- `HAVING` filters groups.
- `ORDER BY` sorts results.

These clauses are extremely important in SQL queries and are frequently used together in real-world database applications.